

StockEx

Developer's Guide

Kafka-based Stock Exchange Simulator
with AI-Powered Clearing House Members

Version 2.0 | March 2026

github.com/Bonum/StockEx

Table of Contents

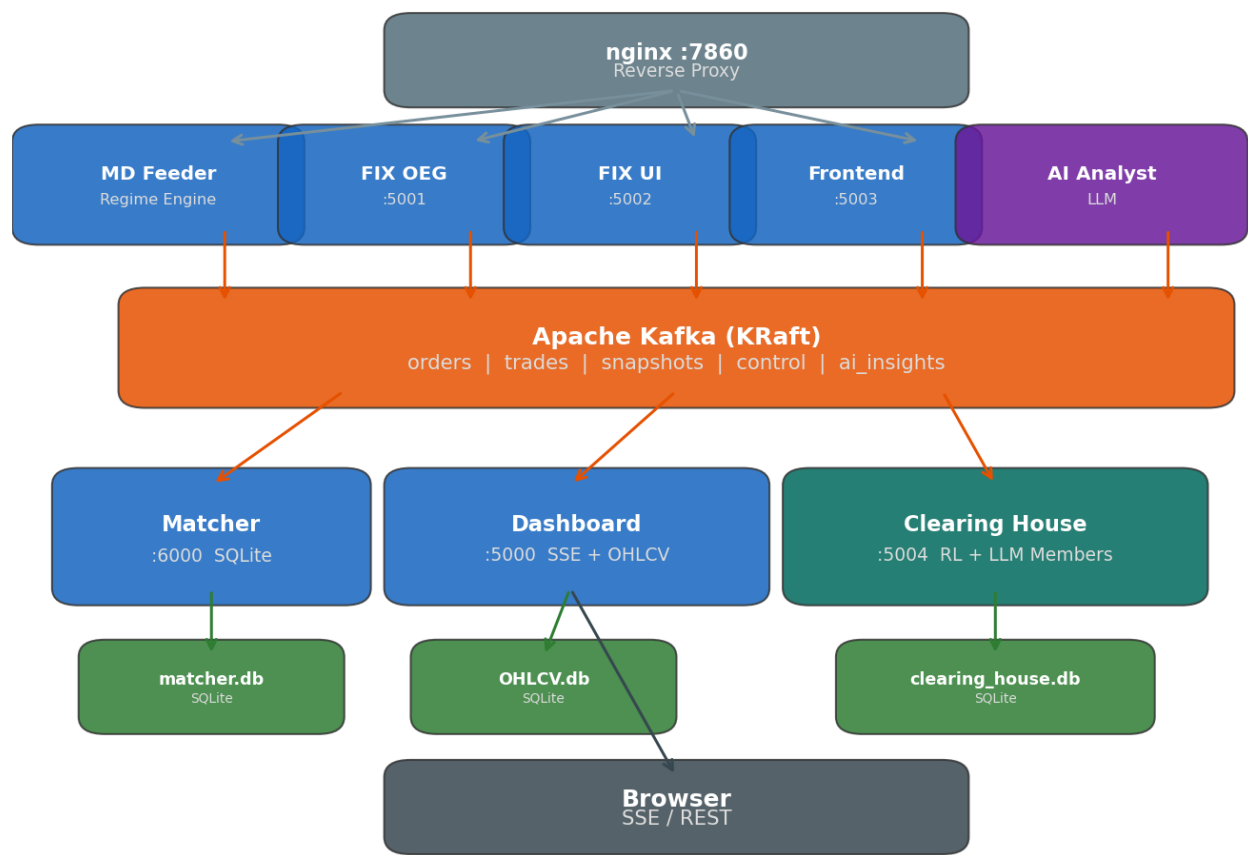
1. System Architecture
2. Kafka Message Flow
3. Services Reference
4. Market Data Feeder -- Regime Engine
5. Clearing House -- AI Trading Members
6. RL Agent -- Neural Network Details
7. LLM Integration
8. Session Lifecycle
9. Database Schema
10. Configuration Reference
11. Deployment

1. System Architecture

StockEx runs as a single Docker container on HuggingFace Spaces (port 7860) or as multiple containers via docker-compose. All services communicate through Apache Kafka (KRaft mode, no ZooKeeper) with five topics: orders, trades, snapshots, control, and ai_insights.

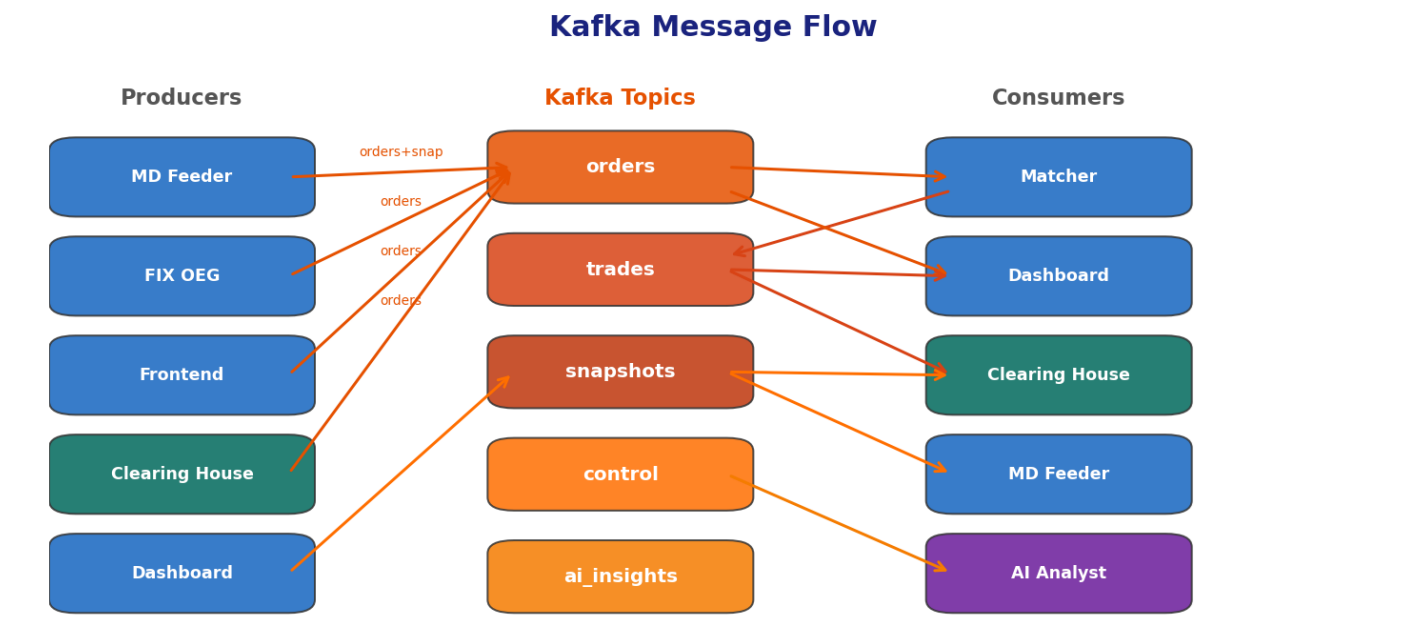
nginx acts as the reverse proxy, routing paths to the appropriate Flask service. Three SQLite databases persist matcher state, OHLCV history, and clearing house data.

System Architecture — Single Container



2. Kafka Message Flow

All order flow and control signals pass through Kafka topics. Producers send messages (orders, snapshots, control signals) and multiple consumers process them independently. This decoupled architecture allows services to be added or removed without affecting others.



Order Message Format

```
{
  "cl_ord_id": "USR01-1709876543210-1",
  "symbol":    "ALPHA",
  "side":      "BUY",
  "quantity":  100,
  "price":     10.50,
  "ord_type":  "LIMIT",
  "time_in_force": "DAY",
  "timestamp": 1709876543.123,
  "source":    "CLRH"
}
```

Trade Message Format

```
{
  "trade_id": 12345,
  "symbol":    "ALPHA",
  "price":     10.50,
  "quantity":  100,
  "buy_id":    "USR01-1709876543210-1",
  "sell_id":   "MDF-1709876543200-42",
  "timestamp": 1709876543.456
}
```

Snapshot with Indicators

```
{
  "symbol": "ALPHA", "best_bid": 24.85, "best_ask": 25.05,
  "bid_size": 200, "ask_size": 150, "timestamp": 1709876543.789,
  "source": "MDF",
  "indicators": {
    "sma_5": 24.92, "sma_20": 24.78,
    "ema_12": 24.88, "ema_26": 24.75,
    "macd": 0.13, "rsi_14": 62.5,
    "bb_pos": 0.72, "regime": "trending_up"
  }
}
```

3. Services Reference

Service	Port	Key File	Description
nginx	7860	nginx.conf	Reverse proxy
Dashboard	5000	dashboard/dashboard.py	SSE streaming, session control, charts
Matcher	6000	matcher/matcher.py	Price-time matching, SQLite persistence
MD Feeder	-	md_feeder/mdf_simulator.py	Regime-based order generation
Clearing House	5004	clearing_house/app.py	AI members, RL/LLM trading
AI Analyst	-	ai_analyst/ai_analyst.py	LLM market commentary
FIX OEG	5001	fix_oeg/fix_oeg_server.py	FIX 4.4 acceptor
FIX UI	5002	fix-ui-client/fix-ui-client.py	Browser FIX client
Frontend	5003	frontend/frontend.py	Order entry form

Startup Order (entrypoint.sh)

Services start sequentially with sleep delays to allow initialization:

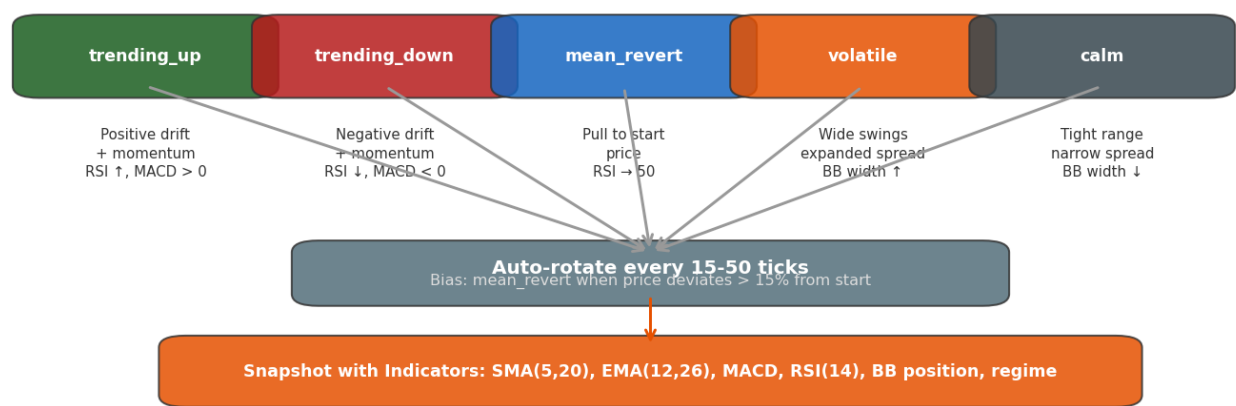
1. Kafka (KRaft) -- wait for broker ready (up to 30 retries)
2. Create Kafka topics (orders, trades, snapshots, control, ai_insights)
3. Matcher (:6000) -- wait 8s
4. MD Feeder -- background
5. FIX OEG (:5001) -- wait 6s
6. FIX UI (:5002) -- wait 3s
7. AI Analyst -- wait 1s
8. Frontend (:5003) -- wait 2s
9. Clearing House (:5004) -- wait 3s
10. nginx (:7860)
11. Dashboard (:5000) -- exec (foreground process)

4. Market Data Feeder -- Regime Engine

The MD Feeder drives price evolution through a PriceDynamics class that simulates five market regimes. Each symbol independently cycles through regimes, producing realistic technical indicator patterns that the RL and LLM strategies can exploit.

Price dynamics use drift + noise + momentum, with adaptive volatility that expands in volatile regimes and contracts in calm periods. When price deviates more than 15% from the session start price, the engine biases regime transitions toward mean reversion.

MD Feeder — Regime-Based Price Dynamics



Regime Parameters

Regime	Drift	Noise	Spread	Aggr. Prob	Indicator Effect
trending_up	+0.2-0.8 * vol	0.5 * vol	0.8-1.5x	20%	MACD > 0, RSI rising
trending_down	-0.2-0.8 * vol	0.5 * vol	0.8-1.5x	20%	MACD < 0, RSI falling
mean_revert	3% pull to start	0.3 * vol	1.0x	20%	RSI oscillates ~50
volatile	random	1.5 * vol	1.5-2.5x	35%	BB width expands
calm	near zero	0.2 * vol	0.6-1.0x	20%	BB width contracts

Order Book Depth

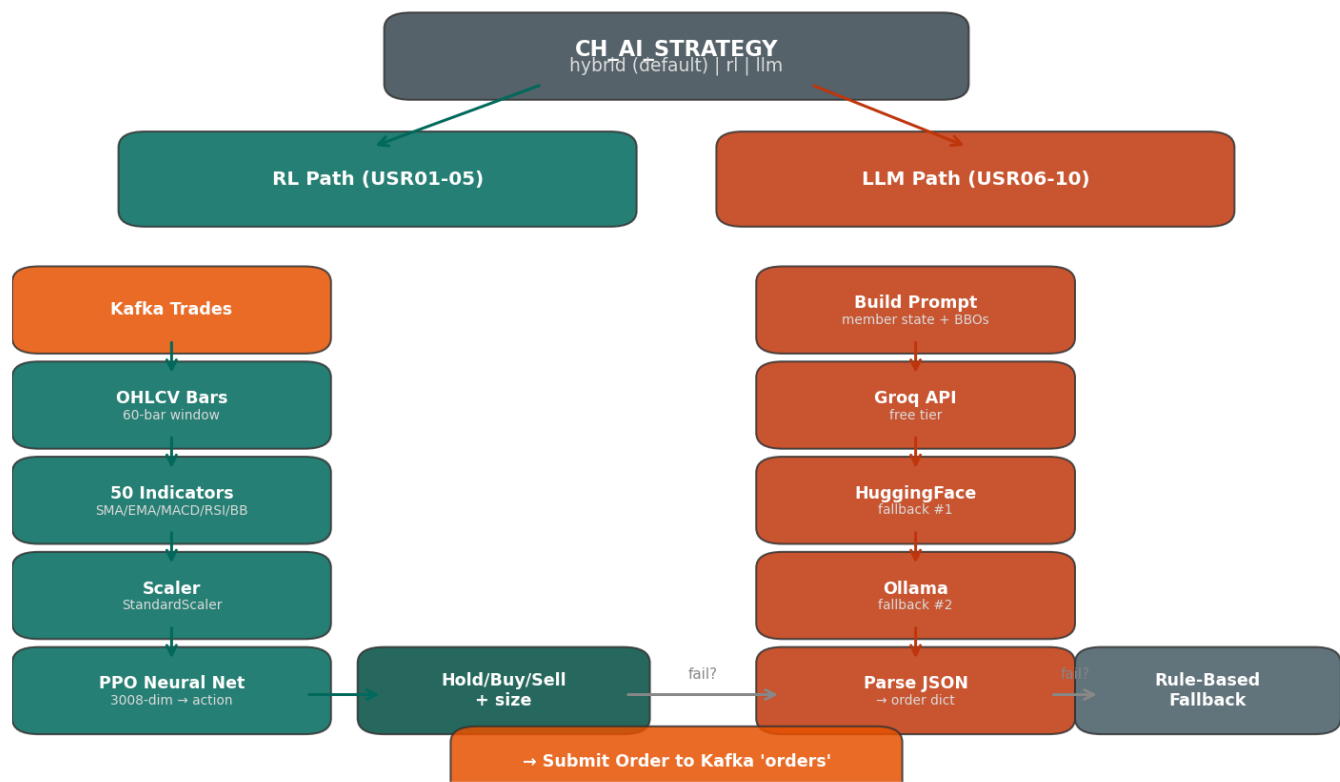
Each cycle places 3 levels of resting bids and asks per symbol (6 passive orders), ensuring the order book always has visible depth. In trending regimes, the passive side gets thicker depth (1.5x quantity) to model institutional support/resistance.

Aggressive orders (20-35% probability) cross the spread to generate trades. The aggressive side is biased by indicators: sell when RSI > 70 + BB overbought, buy when RSI < 30 + BB oversold, and follow MACD direction in trending regimes.

5. Clearing House -- AI Trading Members

The Clearing House service manages 10 simulated trading members (USR01-USR10), each starting with EUR 100,000 capital and a daily obligation to trade at least 20 securities. Members not controlled by a human user are traded by the AI engine.

AI Trading Strategy — Decision Flow



Strategy Selection

Mode	Env Var Value	Behavior
Hybrid (default)	hybrid	USR01-05 = RL, USR06-10 = LLM
RL only	rl	All members use PPO neural network
LLM only	llm	All members use Groq/HF/Ollama

Strategy is set via CH_AI_STRATEGY env var and can be switched at runtime:

```
POST /ch/api/strategy {"strategy": "rl"}
```

Every strategy falls back: RL -> LLM -> rule-based. The rule-based fallback uses portfolio balance heuristics (buy when holdings < 20% of net worth, sell when > 60%).

Trade Attribution

The trade consumer thread monitors the Kafka 'trades' topic. When a trade's buy_id or sell_id matches the pattern USRxx-*, the trade is attributed to that clearing house member. This updates their capital, holdings, and daily trade count in the CH database.

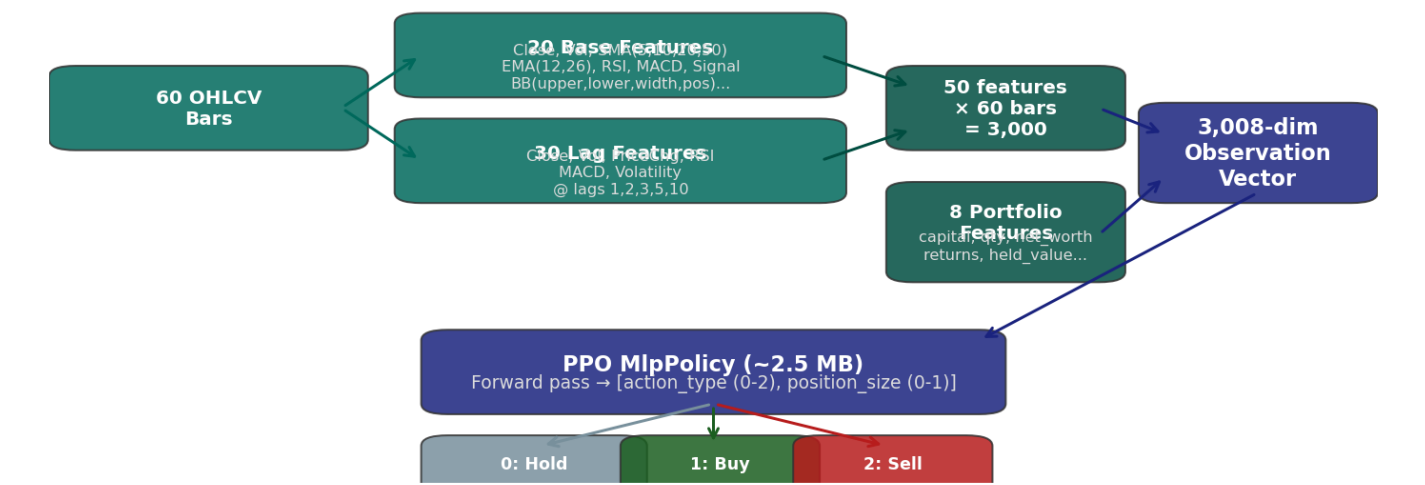
6. RL Agent -- Neural Network Details

The RL strategy uses Adilbai/stock-trading-rl-agent, a PPO (Proximal Policy Optimization) model trained with Stable-Baselines3 on 500,000 timesteps of historical stock data (AAPL, MSFT, GOOGL, AMZN, TSLA).

Model Specifications

Property	Value
Algorithm	PPO (Proximal Policy Optimization)
Policy network	MlpPolicy (Multi-Layer Perceptron)
Model file size	~2.5 MB (final_model.zip)
Scaler file size	~50 KB (scaler.pkl, StandardScaler)
Observation space	3,008 dimensions
Action space	2D: action_type (0-2) + position_size (0-1)
Training steps	500,000
Learning rate	0.0003
Gamma (discount)	0.99
Batch size	64
License	MIT (free, runs locally)
Storage location	/app/data/rl_model/ (downloaded on first use)

RL Agent — Observation Space (3,008 dimensions)



Observation Space Breakdown

The 3,008-dimensional observation vector consists of:

- Market state (3,000 dims): 60 time bars x 50 features per bar
- 20 base features: Close, Volume, SMA(5,10,20,50), EMA(12,26), RSI(14), MACD, Signal, Histogram, BB(upper,lower,width,position), Volatility(20), Price changes, H/L ratio, Volume ratio
 - 30 lag features: Close, Volume, Price change, RSI, MACD, Volatility at lags 1, 2, 3, 5, 10

Portfolio state (8 dims): capital, held_qty, net_worth, returns, held_value, has_position (binary), capital_ratio, holdings_ratio

Price History Management

The RL module (ch_rl_trader.py) maintains a rolling buffer of OHLCV bars per symbol:

- Trades from Kafka feed into a current-bar accumulator (feed_trade())
- Bars finalize every CH_RL_BAR_INTERVAL seconds (default 60s)
- On startup, bars are seeded from BBO reference prices with small noise
- Minimum CH_RL_MIN_BARS (default 30) required before RL predictions start
- Buffer holds up to 120 bars (deque with maxlen)

The shipped scaler.pkl was trained on real stock data. When it fails on synthetic StockEx data (shape mismatch), the code falls back to manual z-score normalization.

Inference vs LLM Comparison

Aspect	RL (PPO)	LLM
Input	3,008-dim float vector	Text prompt (~500 chars)
Processing	NN forward pass (<1ms)	API call (2-30s)
Output	[action_type, position_size]	JSON text to parse
Internet required	No (local model)	Yes (Groq/HF) or Ollama
Cost	Free (CPU inference)	Free tier or self-hosted
Determinism	Configurable	Temperature-dependent

7. LLM Integration

The LLM strategy sends a text prompt containing the member's state (capital, holdings, obligation) and current market BBOs, requesting a single JSON trade decision.

Provider Fallback Chain

Priority (HF Spaces)	Priority (Local)	Provider	Env Vars
1st	2nd	Groq	GROQ_API_KEY, GROQ_MODEL
-	3rd	HuggingFace Inference	HF_TOKEN, HF_MODEL
2nd	1st	Ollama (local)	OLLAMA_HOST, OLLAMA_MODEL

On HuggingFace Spaces (detected via SPACE_ID env var), Groq is preferred (free tier, fast). Locally, Ollama with a fine-tuned model is preferred. The HF Inference endpoint is used as a fallback with the custom fine-tuned model RayMelius/stockex-ch-trader.

If all LLM providers fail, the system falls back to rule-based trading.

Prompt Structure

```
You are simulating clearing house member USR01 making ONE trading decision.

Member state:
  Available capital: EUR 95,230.00
  Securities obligation remaining today: 15 more to trade
  Current holdings:
    ALPHA: 200 shares @ avg cost 24.90

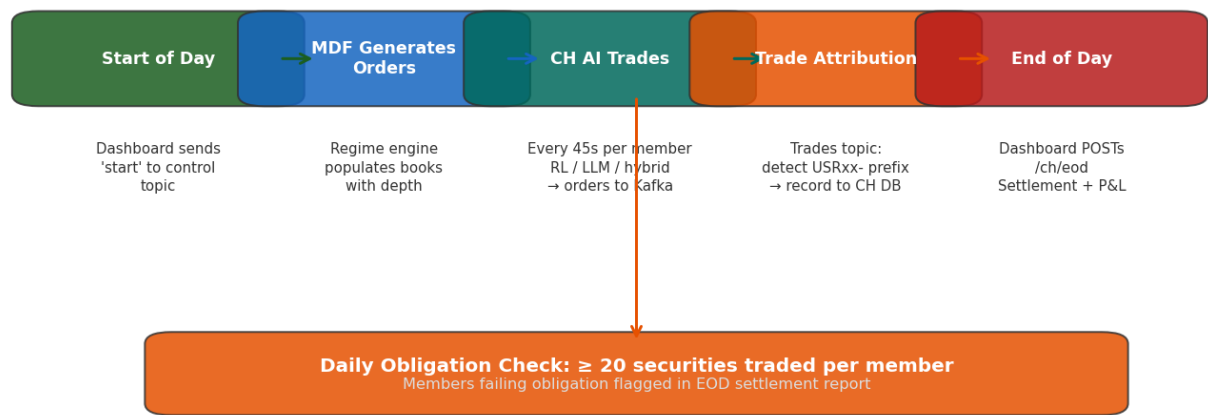
Current market (Bid/Ask):
  ALPHA: Bid 24.85 / Ask 25.05
  NBG: Bid 17.95 / Ask 18.15
  ...

Rules:
- Do not spend more than your available capital
- Do not sell more shares than you hold
- Choose a realistic price close to the BBO mid-price
- Quantity should be between 10 and 200

Respond ONLY with valid JSON:
Example: {"symbol": "ALPHA", "side": "BUY", "quantity": 50, "price": 5.95}
```

8. Session Lifecycle

Clearing House — Session Lifecycle



Control Messages

Action	Dashboard Trigger	Effect on MDF	Effect on CH AI
start	Start of Day button	running=True, reload prices	suspended=False
stop	End of Day button	running=False	suspended=True
suspend	Suspend button	order gen paused	suspended=True
resume	Resume button	order gen resumed	suspended=False

Manual vs Automatic Mode

The Dashboard supports two session modes:

Automatic: A background scheduler reads market_schedule.txt and automatically triggers Start of Day / End of Day based on configured times and timezone.

Manual: The scheduler is disabled. Sessions are started/stopped exclusively via the Dashboard buttons. All trading activity (MDF, CH AI, FIX) still operates normally once a session is started.

EOD Settlement

When End of Day is triggered, the Dashboard POSTs to /ch/eod. The Clearing House:

1. Calculates unrealized P&L for each member (current market value vs avg cost)
2. Checks daily obligation (≥ 20 securities traded)
3. Records settlement entry in the database
4. Logs members who failed their obligation

9. Database Schema

matcher.db (Matcher Service)

Stores all orders and trades for the matching engine.

```
orders:    order_id, symbol, side, price, quantity, remaining_qty,
           status, cl_ord_id, timestamp
trades:    trade_id, symbol, price, quantity, buy_order_id,
           sell_order_id, timestamp
```

clearing_house.db (Clearing House)

Stores member accounts, trade attribution, and settlement history.

```
members:    member_id (PK), password_hash, capital
holdings:    member_id, symbol, quantity, avg_cost
             (composite PK: member_id + symbol)
trade_log:    id, member_id, symbol, side, quantity, price,
             order_id, timestamp
settlements: id, member_id, trading_date, opening_capital,
             closing_capital, realized_pnl, unrealized_pnl,
             obligation_met
ai_decisions: id, member_id, raw_response, parsed_order,
             source, timestamp
```

Dashboard OHLCV (In-Memory + Periodic SQLite)

The Dashboard aggregates trades into 1-minute OHLCV buckets for candlestick charts. Data is held in memory with periodic persistence to SQLite for history.

10. Configuration Reference

Core Settings (shared/config.py)

Variable	Default	Description
KAFKA_BOOTSTRAP	kafka:9092	Kafka broker address
MATCHER_URL	http://matcher:6000	Matcher REST API
SECURITIES_FILE	/app/data/securities.txt	Symbol definitions
TICK_SIZE	0.05	Minimum price increment
ORDERS_PER_MIN	8	MDF order rate per symbol
CH_DAILY_OBLIGATION	20	Min securities per member/day
CH_STARTING_CAPITAL	100,000	Initial member capital (EUR)

AI Strategy Settings

Variable	Default	Description
CH_AI_STRATEGY	hybrid	llm, rl, or hybrid
CH_AI_INTERVAL	45	Seconds between AI cycles
CH_RL_MODEL_REPO	Adilbai/stock-trading-rl-agent	HF model repo
CH_RL_BAR_INTERVAL	60	Seconds per OHLCV bar
CH_RL_MIN_BARS	30	Min bars before RL starts
GROQ_API_KEY	-	Groq API key
GROQ_MODEL	llama-3.1-8b-instant	Groq model name
HF_TOKEN	-	HuggingFace API token
HF_MODEL	RayMelius/stockex-ch-trader	HF model for CH
OLLAMA_HOST	-	Ollama server URL
OLLAMA_MODEL	llama3.1:8b	Ollama model name

11. Deployment

HuggingFace Spaces (Single Container)

The Dockerfile builds a single container with Kafka, all Python services, and nginx. HuggingFace Spaces exposes port 7860. The entrypoint.sh starts services sequentially.

Key dependencies added for RL: stable-baselines3 (pulls PyTorch ~200MB), numpy (<2.0), pandas, scikit-learn, huggingface_hub.

The RL model is downloaded from HuggingFace Hub on first use and cached at /app/data/rl_model/.

Docker Compose (Multi-Container)

docker-compose.yml defines separate containers for each service. The clearing_house service gets its own Dockerfile with the RL dependencies. Volumes persist SQLite databases.

Set CH_AI_STRATEGY in the environment section or .env file.

Local Development

```
# Prerequisites: Python 3.11+, Kafka on localhost:9092
pip install kafka-python Flask requests quickfix \
    stable-baselines3 huggingface_hub pandas scikit-learn

export PYTHONPATH=$(pwd)
export KAFKA_BOOTSTRAP=localhost:9092
export MATCHER_URL=http://localhost:6000
export CH_AI_STRATEGY=hybrid

# Terminal 1-4: matcher, md_feeder, dashboard, clearing_house
```

CI/CD Pipeline

GitHub Actions workflows:

- ci.yml: Run matcher unit tests + Docker build check on every push
- deploy-hf.yml: Auto-deploy to HuggingFace Spaces on push to main (after tests pass)

The container log prints the StockEx version at startup for deployment verification.